

Characterizing Schedules based on Recoverability

Transaction schedule or history:

- When transactions are executing **concurrently in an interleaved fashion**, the **order of execution of operations** from the various transactions forms what is known as a transaction schedule (or history).

A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$:

- It is an **ordering of the operations** of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i .
- Note, however, that operations from other transactions T_j can be interleaved with the operations of T_i in S .

Characterizing Schedules based on Recoverability

Schedules classified on recoverability:

Recoverable schedule:

- One where no transaction needs to be rolled back.
- A schedule S is recoverable **if no transaction T in S commits** until all transactions T' that have written an item that T reads have committed.

Cascadeless schedule:

- One where **every transaction reads only the items that are written by committed transactions.**

Characterizing Schedules based on Recoverability

Schedules classified on recoverability (contd.):

Schedules requiring cascaded rollback:

- A schedule in which **uncommitted transactions that read an item from a failed transaction** must be rolled back.

Strict Schedules:

- A schedule in which a **transaction can neither read or write an item X until the last transaction that wrote X has committed.**

Characterizing Schedules based on Serializability

Serial schedule:

- A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
- Otherwise, the schedule is called nonserial schedule.

Serializable schedule:

- A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.

Characterizing Schedules based on Serializability

Result equivalent:

- Two schedules are called result equivalent if they produce the same final state of the database.

Conflict equivalent:

- Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.

Conflict serializable:

- A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S' .

Characterizing Schedules based on Recoverability

Transaction schedule or history:

- When transactions are executing **concurrently in an interleaved fashion**, **the order of execution of operations** from the various transactions forms what is known as a transaction schedule (or history).

A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$:

- It is an **ordering of the operations** of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i .
- Note, however, that operations from other transactions T_j can be interleaved with the operations of T_i in S .

Characterizing Schedules based on Recoverability

Schedules classified on recoverability:

Recoverable schedule:

- One where no transaction needs to be rolled back.
- A schedule S is recoverable **if no transaction T in S commits** until all transactions T' that have written an item that T reads have committed.

Cascadeless schedule:

- One where **every transaction reads only the items that are written by committed transactions.**

Characterizing Schedules based on Recoverability

Schedules classified on recoverability (contd.):

Schedules requiring cascaded rollback:

- A schedule in which **uncommitted transactions that read an item from a failed transaction** must be rolled back.

Strict Schedules:

- A schedule in which a **transaction can neither read or write an item X until the last transaction that wrote X has committed.**

Characterizing Schedules based on Serializability

Serial schedule:

- A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
- Otherwise, the schedule is called nonserial schedule.

Serializable schedule:

- A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.

Characterizing Schedules based on Serializability

Result equivalent:

- Two schedules are called result equivalent if they produce the same final state of the database.

Conflict equivalent:

- Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.

Conflict serializable:

- A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S' .

Isolation levels in Transaction Processing

A transaction isolation level is defined by the following phenomena –

Dirty Read – A Dirty read is the situation when a transaction reads a data that has not yet been committed.

Non Repeatable read – Non Repeatable read occurs when a transaction reads same row twice, and get a different value each time.

Phantom Read – Phantom Read occurs when two same queries are executed, but the rows retrieved by the two, are different.

Based on these phenomena, The SQL standard defines four isolation levels :

Isolation levels in Transaction Processing

Read Uncommitted – Read Uncommitted is the lowest isolation level. In this level, one transaction may read not yet committed changes made by other transaction, thereby allowing dirty reads. In this level, transactions are not isolated from each other.

Read Committed – This isolation level guarantees that any data read is committed at the moment it is read. Thus it does not allow dirty read. The transaction holds a read or write lock on the current row, and thus prevent other transactions from reading, updating or deleting it.

Isolation levels in Transaction Processing

Repeatable Read – This is the most restrictive isolation level. The transaction holds read locks on all rows it references and writes locks on all rows it inserts, updates, or deletes. Since other transaction cannot read, update or delete these rows, consequently it avoids non-repeatable read.

Serializable – This is the Highest isolation level. A *serializable* execution is guaranteed to be serializable. Serializable execution is defined to be an execution of operations in which concurrently executing transactions appears to be serially executing.

Thank you